

QuikVote API Specification

CSI 4160

Gavin D'Hondt, Ricky Hodge, Adam Walsh

2026-02-18

API Overview and Design

The QuikVote API is designed as a stateless RESTful JSON web service that enables rapid poll creation, flexible voting mechanisms, and efficient consensus building for groups. Its primary function is to allow users to create polls instantly, cast weighted votes, and compute results using enhanced reconciliation logic such as weighted scoring and tie-breaking.

Design Principles

The QuikVote API attempts to follow the best modern API design principles:

- **Stateless requests:** each request works in a vacuum and respects idempotency
- **Stateful resources:** polls and votes are persisted and evolve over time
- **RESTful semantics:** resources are exposed according to their use case and following idiomatic REST according to HTTP verb and path
- **Semantic errors:** error codes match uses cases and intuitive usage
- **Strong typing:** request and response schemas are strictly defined and follow strongly typed schemas to avoid confusion or mishandling

Authentication

The API is stateless with respect to user identity. No user accounts or sessions are stored. Each request must include all required identifiers. Poll data itself is persisted in the database for retrieval and vote reconciliation.

No account-based authentication is required. Polls may optionally be password-protected for controlled access

API Specification

POST /polls

Description: Creates a new poll with options and voting configuration.

Request Schema (JSON)

Request body required application/json

Example Value | Schema

CreatePollRequest {
 title*
 description*
 options*
 visibility*
 password
 allow_weighted_voting*
 enable_veto*
 use_llm_option_generation*
}

string
minLength: 1

string
minLength: 1

[
 string
 minLength: 1
]

Visibility
Enum:
[public, password_protected]

string
minLength: 1
Required when visibility is password_protected.

boolean

boolean

boolean

Request body required application/json

Example Value | Schema

```
{  
  "title": "Lunch plan",  
  "description": "Team lunch this Friday",  
  "options": [  
    "Pizza",  
    "Sushi"  
  ],  
  "visibility": "public",  
  "allow_weighted_voting": true,  
  "enable_veto": true,  
  "use_llm_option_generation": true  
}
```

Response Schema (JSON)

Responses		
Code	Description	Links
201	Poll created Media type application/json Controls Accept header. Example Value Schema <pre>{ "poll_id": "string", "title": "string", "description": "string", "options": [{ "option_id": "string", "text": "string", "total_weighted_score": 0, "veto_count": 0 }], "visibility": "public", "created_at": "2026-02-18T01:25:17.453Z", "status": "active"}</pre>	No links
400	Validation error Media type application/json Example Value Schema <pre>{ "error": "string"}</pre>	No links

POST /polls/{poll_id}/vote

Description: Submits a vote to a specific option in a poll.

Request Schema (JSON)

Parameters

Try it out

Name	Description
poll_id * required string (path)	Unique poll identifier

Request body required

application/json

Example Value

Schema

VoteSubmissionRequest

selected_option_id*

weight

veto

password

string

integer

boolean

string

minLength: 1

minimum: 1

default: false

Required when weighted voting is enabled for the poll.

Required for password-protected polls.

Parameters

Try it out

Name	Description
poll_id * required string (path)	Unique poll identifier

Request body required

application/json

Example Value

Schema

{

"selected_option_id": "opt_1234abcd",

"weight": 4,

"veto": false

}

Response Schema (JSON)

Responses		
Code	Description	Links
200	Vote accepted	No links
Media type application/json Controls Accept header. Example Value Schema <pre>{ "message": "string", "poll_id": "string", "selected_option_id": "string", "applied_weight": 5, "veto_applied": true, "updated_option": { "option_id": "string", "text": "string", "total_weighted_score": 0, "veto_count": 0 }, "results_summary": { "poll_id": "string", "winning_option": { "option_id": "string", "text": "string", "total_weighted_score": 0, "veto_count": 0 }, "ranked_result": [{ "option_id": "string", "text": "string", "total_weighted_score": 0 }] } }</pre>		
400	Validation error	No links
Media type application/json Example Value Schema <pre>{ "error": "string" }</pre>		
401	Missing or invalid password	No links
Media type application/json Example Value Schema <pre>{ "error": "string" }</pre>		
404	Poll or option not found	No links
Media type application/json Example Value Schema <pre>{ "error": "string" }</pre>		
409	Poll is closed	No links
Media type application/json Example Value Schema <pre>{ "error": "string" }</pre>		

GET /polls/{poll_id}

Description: Retrieves poll details and, by default, includes computed results.

Request Schema (JSON)

Parameters

Try it out

Name	Description
poll_id * required string (path)	Unique poll identifier
include_results boolean (query)	If false, omit results from response. <i>Default value : true</i>
password string (query)	Required for password-protected polls.

Response Schema (JSON)

Responses		
Code	Description	Links
200	Poll returned Media type application/json Controls Accept header. Example Value Schema <pre>{ "poll": { "poll_id": "string", "title": "string", "description": "string", "options": [{ "option_id": "string", "text": "string", "total_weighted_score": 0, "veto_count": 0 }], "visibility": "public", "created_at": "2026-02-18T01:48:50.751Z", "status": "active" }, "results": { "poll_id": "string", "winning_option": { "option_id": "string", "text": "string", "total_weighted_score": 0, "veto_count": 0 } }}</pre>	No links
401	Missing or invalid password Media type application/json Example Value Schema <pre>{ "error": "string"}</pre>	No links
404	Poll not found Media type application/json Example Value Schema <pre>{ "error": "string"}</pre>	No links

POST /polls/{poll_id}/close

Description: Closes a poll and returns finalized results.

Request Schema (JSON)

Parameters

Try it out

Name	Description
poll_id ★ required string <i>(path)</i>	Unique poll identifier

Request body

application/json

Example Value

Schema

ClosePollRequest

password

string

Required for password-protected polls.

Parameters

Try it out

Name	Description
poll_id ★ required string <i>(path)</i>	Unique poll identifier

Request body

application/json

Example Value

Schema

```
{
  "password": "secret123"
}
```

Response Schema (JSON)

Responses		
Code	Description	Links
200	Poll closed Media type application/json Controls Accept header. Example Value Schema <pre>{ "poll": { "poll_id": "string", "title": "string", "description": "string", "options": [{ "option_id": "string", "text": "string", "total_weighted_score": 0, "veto_count": 0 }], "visibility": "public", "created_at": "2026-02-18T01:49:36.970Z", "status": "active" }, "finalized_results": { "poll_id": "string", "winning_option": { "option_id": "string", "text": "string", "total_weighted_score": 0, "veto_count": 0 } },}</pre>	No links
401	Missing or invalid password Media type application/json Example Value Schema <pre>{ "error": "string"}</pre>	No links
404	Poll not found Media type application/json Example Value Schema <pre>{ "error": "string"}</pre>	No links

Architecture and Workflow

Workflow Steps

1. **Request Initiation:** The client application sends a request (e.g., create poll, submit vote, retrieve results) to the appropriate endpoint with all required identifiers and data
2. **Poll Creation and LLM Option Generation:** If enabled, the API server constructs a prompt and sends it to a Large Language Model (Gemini) to generate intelligent or creative additional options based on the poll title and description. The generated options are merged with user-provided options before persistence
3. **Vote Processing and Reconciliation:** When a vote is submitted, the API:
 - a. Validates poll status and password
 - b. Applies weighted scoring if enabled
 - c. Registers vetos if enabled
 - d. Updates aggregated totals in the database
4. **Tie Breaking and Winner Determination:** Upon poll closure or result request:
 - a. The server ranks options by weighted score
 - b. If veto logic disqualifies an option, it is removed from contention
 - c. If a tie remains, a randomness-based selection algorithm determines the winner
 - d. The system generates a truthful result_justification explaining how the decision was reached
5. **Response Assembly:** The server formats the poll data and computed results into the defined response structures and returns them to the client application.

Tech Stack Justification

We chose a microservice-based design to have a distinct separation between subsystems in our application, which allows for independent scaling of each service and fault isolation. It also creates clear enforceable boundaries between systems to ensure the systems are built in a maintainable, high quality manner.

Cloud Infrastructure

AWS - Industry standard for serverless computing, managed API routing, monitoring, and scalability

AWS CDK - Enables long term maintainability, reproducibility, and safety of infrastructure management and deployment

QuikVote Website

React - Industry standard for its separation of concerns, ease of development, and component-based architecture

TypeScript - Enables safe frontend development with a strict typing system

Vite - Enables fast development, server start up, and high quality bundling to minimal artifact payloads

TanStack - Best modern library for connecting a frontend React app to REST APIs

Voting Service

Python - Readability and ease of development

FastAPI - Built in request validation and OpenAPI documentation generation

AWS Lambda - Enables serverless architecture, auto scaling, and automatic deployments

Magnum - Bridges FastAPI with AWS Lambda which also enables serverless deployment

Backend-to-Frontend (BFF) Pattern - BFF allows for aggregated responses and reduced fetching

Notification Service

Twilio - Industry standard, provides reliable and globally scalable SMS messaging

SendGrid - Industry standard, provides scalable and reliable email delivery

Suggestion Service

Google Gemini - Enables QwikVote be more than just a simple vote counting application:
enables us to have ad-hoc, human-like intelligence

Randomness Service

Random.org - Provides true randomness compared to a base random function in Python, which adds more fairness into the tie breaking process